

# AI Story Generator API 명세서

현재 FastAPI 애플리케이션(`app/main.py`) 기준의 API 명세입니다.

## 기본 정보

- 기본 URL (로컬): `http://127.0.0.1:8000`
- 기본 URL (외부): `https://mystorybook.loca.lt`
- Swagger UI: `http://127.0.0.1:8000/docs`
- 응답 포맷: JSON, SSE, PNG, WAV
- CORS: 모든 origin, method, header 허용

## 외부 접속 (Localtunnel)

### 구성

`START_ALL.bat` 실행 시 Localtunnel이 자동으로 시작됩니다.

| 항목        | 값                                                 |
|-----------|---------------------------------------------------|
| 터널 URL    | <code>https://mystorybook.loca.lt</code>          |
| 마스터 공인 IP | <code>59.3.103.182</code>                         |
| 포트 포워딩 대상 | <code>localhost:8000</code>                       |
| 재연결       | 끊기면 3초 후 자동 재시작 ( <code>start_tunnel.ps1</code> ) |

**서브도메인 주의:** `mystorybook` 서브도메인이 이미 사용 중이면 랜덤 URL이 할당됩니다. 터널 터미널 창에서 실제 URL을 확인하세요.

### 브라우저 첫 접속 시 우회 페이지

loca.lt 터널은 브라우저에서 처음 열면 **"Tunnel Unavailable"** 또는 우회 확인 페이지가 표시됩니다.

**해결 방법 1** — URL에 파라미터 추가:

```
https://mystorybook.loca.lt/?bypass-tunnel-reminder=true
```

**해결 방법 2** — 우회 페이지에서 공인 IP 입력:

```
59.3.103.182
```

API 클라이언트(curl, fetch 등)는 우회 페이지 없이 바로 요청 가능합니다.

### curl 예시 (외부에서)

```
# 헬스 체크
curl https://mystorybook.localt/health

# 작업 생성
curl -X POST https://mystorybook.localt/api/runs \
  -H "Content-Type: application/json" \
  -d '{
    "era_ko": "조선 시대",
    "place_ko": "한양",
    "characters_ko": "호기심 많은 아이와 말하는 호랑이",
    "topic_ko": "우정"
  }'

# 상태 조회
curl https://mystorybook.localt/api/runs/20260429_173423_d118aa
```

## 접속 URL 정리

| 환경             | URL                        |
|----------------|----------------------------|
| 로컬 (마스터 서버 내부) | http://127.0.0.1:8000      |
| LAN (같은 네트워크)  | http://<마스터_내부_IP>:8000    |
| 외부 인터넷         | https://mystorybook.localt |

## 공통 에러 응답

FastAPI 기본 검증 오류:

```
{
  "detail": [
    {
      "type": "missing",
      "loc": ["body", "era_ko"],
      "msg": "Field required",
      "input": {}
    }
  ]
}
```

HTTPException 오류:

```
{
  "detail": "Run 20260429_173423_d118aa not found"
}
```

## 엔드포인트 요약

| Method | Path                                 | 설명                                                  |
|--------|--------------------------------------|-----------------------------------------------------|
| GET    | /                                    | 정적 UI( <code>static/index.html</code> ) 또는 서버 상태 반환 |
| GET    | /health                              | 헬스 체크                                               |
| POST   | /api/stt/field                       | 음성 파일을 특정 입력 필드 텍스트로 변환                             |
| POST   | /api/runs                            | 동화 생성 작업 시작                                         |
| GET    | /api/runs/{run_id}                   | 생성 작업 상태 조회                                         |
| GET    | /api/runs/{run_id}/events            | 생성 작업 상태 SSE 스트림                                    |
| GET    | /api/runs/{run_id}/story             | 생성된 원본 story JSON 다운로드                              |
| GET    | /api/runs/{run_id}/images/{filename} | 생성 이미지 다운로드                                         |
| GET    | /api/runs/{run_id}/audio/{filename}  | 생성 오디오 다운로드                                         |

### GET /

정적 UI가 있으면 `static/index.html`을 반환합니다. 없으면 JSON 상태를 반환합니다.

#### 응답 예시

```
{
  "status": "ok",
  "version": "3.0.0"
}
```

### GET /health

서버 상태 확인용 엔드포인트입니다.

#### 응답

```
{
  "status": "ok"
}
```

### POST /api/stt/field

업로드한 음성 파일을 Whisper로 텍스트 변환하고, 특정 입력 필드 값으로 사용할 문자열을 반환합니다.

## Request

Content-Type: `multipart/form-data`

| 필드                      | 타입     | 필수 | 설명                                                                                                  |
|-------------------------|--------|----|-----------------------------------------------------------------------------------------------------|
| <code>audio_file</code> | file   | O  | 음성 파일. <code>webm</code> , <code>mp4</code> , <code>mp3</code> , <code>wav</code> 등                 |
| <code>field_type</code> | string | O  | 변환 대상 필드. <code>era</code> , <code>place</code> , <code>characters</code> , <code>topic</code> 중 하나 |
| <code>language</code>   | string | X  | 언어 코드. 기본값 <code>ko-KR</code>                                                                       |

## 성공 응답

Status: `200 OK`

```
{
  "stt_text": "조선 시대",
  "parsed_value": "조선 시대",
  "confidence": 0.91
}
```

## 응답 필드

| 필드                        | 타입     | 설명                             |
|---------------------------|--------|--------------------------------|
| <code>stt_text</code>     | string | Whisper 원문 인식 결과               |
| <code>parsed_value</code> | string | 입력 필드에 넣을 정리된 값                |
| <code>confidence</code>   | number | 인식 신뢰도. <code>0.0 ~ 1.0</code> |

## 에러

- `400 Bad Request`: `field_type` 값이 유효하지 않음
- `400 Bad Request`: 빈 음성 파일
- `500 Internal Server Error`: STT 처리 실패

## curl 예시

```
curl -X POST http://127.0.0.1:8000/api/stt/field \
  -F "audio_file=@sample.wav" \
  -F "field_type=topic"
```

## POST /api/runs

동화 생성 작업을 생성하고 백그라운드 파이프라인을 시작합니다.

파이프라인은 두 머신이 병렬로 동작합니다.

- **워커 (RTX 5080):** LLM 스토리 생성 → 표지 TTS + 모든 Scene TTS → 워커 배정 Scene 이미지
- **마스터 (RTX 3080 Ti):** 표지 이미지 생성 → 마스터 배정 Scene 이미지

현재 이미지 분배:

| 처리 위치             | 생성 이미지                                                                                                                                                       |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 워커 (RTX 5080)     | scene_02_img_01.png, scene_02_img_03.png, scene_03_img_02.png, scene_04_img_01.png, scene_04_img_03.png                                                      |
| 마스터 (RTX 3080 Ti) | cover.png, scene_01_img_01.png, scene_01_img_02.png, scene_01_img_03.png, scene_02_img_02.png, scene_03_img_01.png, scene_03_img_03.png, scene_04_img_02.png |

음성 생성은 현재 워커에서만 수행합니다. 마스터는 TTS 모델을 로드하거나 음성을 생성하지 않습니다.

Request

Content-Type: application/json

```
{
  "era_ko": "조선 시대",
  "place_ko": "한양의 작은 서당",
  "characters_ko": "호기심 많은 아이와 말하는 호랑이",
  "topic_ko": "서로를 믿는 우정"
}
```

요청 필드

| 필드            | 타입     | 필수 | 설명   |
|---------------|--------|----|------|
| era_ko        | string | O  | 시대   |
| place_ko      | string | O  | 장소   |
| characters_ko | string | O  | 등장인물 |
| topic_ko      | string | O  | 주제   |

성공 응답

Status: 201 Created

```
{
  "run_id": "20260429_173423_d118aa"
}
```

## 처리 결과 저장 위치

```

outputs/runs/{run_id}/
  story.json
  events.jsonl
  images/
    cover.png
    scene_01_img_01.png ~ scene_04_img_03.png
  audio/
    cover.wav
    scene_01.wav ~ scene_04.wav

```

## GET /api/runs/{run\_id}

작업의 현재 상태와 생성된 산출물 URL을 조회합니다.

### Path Parameters

| 이름     | 타입     | 설명                        |
|--------|--------|---------------------------|
| run_id | string | POST /api/runs에서 받은 작업 ID |

### 성공 응답

Status: 200 OK

```

{
  "run_id": "20260429_173423_d118aa",
  "status": "RUNNING",
  "stage": "PARALLEL",
  "story_title": "말하는 호랑이와 작은 악속",
  "cover_image_url": "/api/runs/20260429_173423_d118aa/images/cover.png",
  "cover_audio_url": "/api/runs/20260429_173423_d118aa/audio/cover.wav",
  "scenes": [
    {
      "scene_no": 1,
      "status": "End",
      "scenarios": [
        {
          "index": 0,
          "msg": "",
          "audio_url": "",
          "image_url":
            "/api/runs/20260429_173423_d118aa/images/scene_01_img_01.png",
          "delay": 4
        },
        {
          "index": 1,
          "msg": "한양의 작은 서당 앞에 커다란 발자국이 남아 있었습니다.",

```

```

    "audio_url": "/api/runs/20260429_173423_d118aa/audio/scene_01.wav",
    "image_url": "",
    "delay": 0
  },
  {
    "index": 2,
    "msg": "",
    "audio_url": "",
    "image_url":
"/api/runs/20260429_173423_d118aa/images/scene_01_img_02.png",
    "delay": 4
  },
  {
    "index": 3,
    "msg": "",
    "audio_url": "",
    "image_url":
"/api/runs/20260429_173423_d118aa/images/scene_01_img_03.png",
    "delay": 4
  },
  {
    "index": 4,
    "msg": "누가 이렇게 큰 발자국을 남긴 걸까?",
    "audio_url": "",
    "image_url": "",
    "delay": 0
  }
]
},
"error": null
}

```

## 응답 필드

| 필드              | 타입          | 설명                                           |
|-----------------|-------------|----------------------------------------------|
| run_id          | string      | 작업 ID                                        |
| status          | string      | QUEUED, RUNNING, DONE, FAILED 중 하나           |
| stage           | string      | 현재 단계. LLM, TTS, IMAGE, PARALLEL 중 하나        |
| story_title     | string      | 생성된 동화 제목. LLM 완료 전에는 빈 문자열                  |
| cover_image_url | string      | 표지 이미지 URL. 생성 전에는 빈 문자열                     |
| cover_audio_url | string      | 표지 오디오 URL. 생성 전에는 빈 문자열                     |
| scenes          | array       | 장면 목록. <b>항상 4개 고정</b> . LLM 완료 전에는 빈 배열([]) |
| error           | string null | 실패 시 에러 메시지                                  |

**Cover 분리 이유:** Cover는 동화 전체 표지로 앱 진입 시 독립적으로 표시되는 요소입니다. 이미지 1개 + 제목  
 낭독 오디오 1개로 구성되며, 스토리 씬 재생 흐름과 별개로 사용됩니다.

## scenes[] 필드

| 필드        | 타입     | 설명                           |
|-----------|--------|------------------------------|
| scene_no  | number | 장면 번호. 1~4                   |
| status    | string | Pending, Running, End 중 하나   |
| scenarios | array  | 재생 시퀀스 목록. 항상 5개 (index 0~4) |

## scenes[].status 값

| 값       | 조건                 |
|---------|--------------------|
| Pending | 오디오·이미지 모두 미생성     |
| Running | 일부 리소스 생성 완료       |
| End     | 오디오 + 이미지 3개 모두 완료 |

## scenarios[] 필드

| 필드        | 타입     | 설명                                               |
|-----------|--------|--------------------------------------------------|
| index     | number | 재생 순서. 0~4                                       |
| msg       | string | 자막 텍스트. index 1 = 내레이션, index 4 = 대사. 나머지는 빈 문자열 |
| audio_url | string | 오디오 URL. index 1에만 존재. 나머지는 빈 문자열                |
| image_url | string | 이미지 URL. index 0·2·3에만 존재. 나머지는 빈 문자열            |
| delay     | number | 다음 index로 넘어가기 전 대기 시간(초). 아래 설명 참고              |

## delay 동작 규칙

| delay 값 | 동작                                                 |
|---------|----------------------------------------------------|
| 0       | 오디오 재생 완료까지 대기 (index 1) 또는 자막만 표시 (index 4)       |
| N (양수)  | 이미지 표시 후 N초 대기. <b>WAV 재생시간 ÷ 3 반올림</b> 값으로 자동 계산됨 |

**delay 계산 예시:** WAV 재생시간이 12초이면  $\text{delay} = \text{round}(12 / 3) = 4$ . 최솟값은 1초.

## 씬 재생 흐름

```

index 0 → img_01 표시, delay초 대기
index 1 → 오디오 재생 시작 (내레이션 → 0.5초 무음 → 대사 순서로 합성됨)
           내레이션 자막 표시, 오디오 완료까지 대기
index 2 → img_02로 전환, delay초 대기
  
```

index 3 → img\_03으로 전환, delay초 대기  
 index 4 → 대사 자막 표시 (별도 오디오 없음)

**오디오 구조:** 씬당 WAV 1개에 내레이션과 대사가 순서대로 합성됩니다 (내레이션 → 0.5초 무음 → 대사). index 4의 대사 자막은 오디오가 이미 재생 완료된 후 표시됩니다.

## 상태 값

| 값       | 설명         |
|---------|------------|
| QUEUED  | 작업 생성 직후   |
| RUNNING | 파이프라인 실행 중 |
| DONE    | 모든 처리 완료   |
| FAILED  | 처리 실패      |

## 단계 값

| 값        | 설명                                                 |
|----------|----------------------------------------------------|
| LLM      | 워커에서 스토리 JSON 생성 중                                 |
| PARALLEL | 워커·마스터 병렬 실행 중. 워커는 TTS와 배경 이미지, 마스터는 표지/배경 이미지 처리 |
| IMAGE    | 파이프라인 완료 시 최종 단계 또는 파일 복구 시 이미지 산출물이 있는 상태         |
| TTS      | 하위 호환용 enum 값. 현재 파이프라인에서는 직접 설정하지 않음              |

## 감정 코드

TTS에 전달되는 스타일 매핑:

| 감정 코드 | TTS 스타일 프롬프트            |
|-------|-------------------------|
| 행복    | cheerful tone           |
| 슬픔    | sad emotional voice     |
| 화남    | angry aggressive tone   |
| 밝음    | bright energetic tone   |
| 긴장    | nervous hesitant voice  |
| 무서움   | fearful trembling voice |

## 에러

- 404 Not Found: 작업 상태가 메모리에 없고 파일에서도 복구할 수 없음

**참고:** 작업 상태는 실행 중에는 인메모리 레지스트리에 저장됩니다. 서버를 재시작한 뒤에는 `outputs/runs/{run_id}`의 `story.json`, 이미지, 오디오 파일을 기준으로 가능한 상태를 복구합니다.

## GET /api/runs/{run\_id}/events

작업 진행 상황을 Server-Sent Events(SSE)로 스트리밍합니다.

성공 응답

Content-Type: `text/event-stream`

이벤트 타입:

| 이벤트                    | 설명                    |
|------------------------|-----------------------|
| <code>update</code>    | 작업 상태 변경 또는 산출물 생성 알림 |
| <code>keepalive</code> | 연결 유지용 빈 이벤트 (30초마다)  |

### update 이벤트 기본 필드

| 필드                  | 타입     | 설명       |
|---------------------|--------|----------|
| <code>run_id</code> | string | 작업 ID    |
| <code>status</code> | string | 현재 작업 상태 |
| <code>stage</code>  | string | 현재 처리 단계 |

### 상황별 추가 필드

| 필드                           | 타입     | 발생 시점                                           |
|------------------------------|--------|-------------------------------------------------|
| <code>cover_image_url</code> | string | 표지 이미지 생성 완료 시                                  |
| <code>cover_audio_url</code> | string | 표지 TTS 완료 시                                     |
| <code>scene_no</code>        | number | 장면 이미지·오디오 완료 시                                 |
| <code>image_url</code>       | string | 특정 장면 이미지 생성 완료 시 ( <code>scene_no</code> 와 함께) |
| <code>audio_url</code>       | string | 특정 장면 TTS 완료 시 ( <code>scene_no</code> 와 함께)    |
| <code>error</code>           | string | <code>status</code> 가 <code>FAILED</code> 일 때   |

### 이벤트 흐름 예시

```
event: update
data: {"run_id":"...", "status":"RUNNING", "stage":"LLM"}

event: update
data: {"run_id":"...", "status":"RUNNING", "stage":"PARALLEL"}

event: update
data:
{"run_id":"...", "status":"RUNNING", "stage":"PARALLEL", "cover_audio_url":"/api/runs
```

```

/.../audio/cover.wav"}

event: update
data:
{"run_id":"...", "status":"RUNNING", "stage":"PARALLEL", "scene_no":1, "audio_url":"/a
pi/runs/.../audio/scene_01.wav"}

event: update
data:
{"run_id":"...", "status":"RUNNING", "stage":"PARALLEL", "cover_image_url":"/api/runs
/.../images/cover.png"}

event: update
data:
{"run_id":"...", "status":"RUNNING", "stage":"PARALLEL", "scene_no":2, "image_url":"/a
pi/runs/.../images/scene_02_img_01.png"}

event: update
data: {"run_id":"...", "status":"DONE", "stage":"IMAGE"}

event: keepalive
data:

```

## 브라우저 예시

```

const source = new EventSource("/api/runs/20260429_173423_d118aa/events");

source.addEventListener("update", (e) => {
  const data = JSON.parse(e.data);
  if (data.status === "DONE") source.close();
});

```

## 에러

- **404 Not Found**: 작업 상태가 메모리에 없고 파일에서도 복구할 수 없음

## GET /api/runs/{run\_id}/story

LLM이 생성한 원본 **story.json**을 반환합니다.

## 성공 응답

Content-Type: **application/json**

```

{
  "title": "말하는 호랑이와 작은 악속",
  "cover_prompt": "children's book cover, ...",
  "scenes": [

```

```
{
  "scene_no": 1,
  "narration": "한양의 작은 서당 앞에 커다란 발자국이 남아 있었습니다.",
  "dialogue": "누가 이렇게 큰 발자국을 남긴 걸까?",
  "narration_emotion": "밝음",
  "dialogue_emotion": "긴장",
  "image_prompts": [
    "watercolor children's book illustration, ...",
    "watercolor children's book illustration, ...",
    "watercolor children's book illustration, ..."
  ]
}
```

### story.json 스키마

| 필드           | 타입     | 제약                 |
|--------------|--------|--------------------|
| title        | string | 동화 전체 제목. 빈 문자열 불가 |
| cover_prompt | string | 빈 문자열 불가           |
| scenes       | array  | 정확히 4개             |

### scenes[] 스키마

| 필드                | 타입       | 제약                  |
|-------------------|----------|---------------------|
| scene_no          | number   | 1~4, 순서대로 증가        |
| narration         | string   | 빈 문자열 불가            |
| dialogue          | string   | 빈 문자열 불가            |
| narration_emotion | string   | 허용 감정 코드 중 하나       |
| dialogue_emotion  | string   | 허용 감정 코드 중 하나       |
| image_prompts     | string[] | 정확히 3개, 빈 문자열·한자 불가 |

### 예러

- 404 Not Found: 작업 상태가 없거나 story.json이 아직 생성되지 않음

## GET /api/runs/{run\_id}/images/{filename}

생성된 PNG 이미지를 반환합니다.

### Path Parameters

| 이름                    | 타입     | 설명      |
|-----------------------|--------|---------|
| <code>run_id</code>   | string | 작업 ID   |
| <code>filename</code> | string | 이미지 파일명 |

### 파일명 패턴

```
cover.png
scene_01_img_01.png ~ scene_04_img_03.png
```

### 성공 응답

- Status: `200 OK`
- Content-Type: `image/png`
- Header: `Cache-Control: no-store`

### 에러

- `400 Bad Request`: 파일명에 `..`, `/`, `\` 포함
- `404 Not Found`: 이미지 파일 없음

---

## GET `/api/runs/{run_id}/audio/{filename}`

생성된 WAV 오디오를 반환합니다.

### Path Parameters

| 이름                    | 타입     | 설명      |
|-----------------------|--------|---------|
| <code>run_id</code>   | string | 작업 ID   |
| <code>filename</code> | string | 오디오 파일명 |

### 파일명 패턴

```
cover.wav          ← 표지 TTS (동화 제목 낭독)
scene_01.wav ~ scene_04.wav ← 씬 TTS (내레이션 → 0.5초 무음 → 대사 순서로 합성)
```

### 오디오 형식

| 항목   | 값                   |
|------|---------------------|
| 컨테이너 | WAV (RIFF PCM, 비압축) |
| 채널   | 모노                  |

| 항목    | 값                        |
|-------|--------------------------|
| 샘플레이트 | TTS 모델 native rate       |
| 전송 방식 | 파일 전체 다운로드 (Range 요청 지원) |

## 성공 응답

- Status: 200 OK
- Content-Type: audio/wav
- Header: Cache-Control: no-store

## 에러

- 400 Bad Request: 파일명에 .., /, \ 포함
- 404 Not Found: 오디오 파일 없음

## 클라이언트 호출 흐름

1. POST /api/runs로 작업 생성 → run\_id 수신
2. GET /api/runs/{run\_id}/events SSE 연결 (또는 2500ms 폴링)
3. scenes.length === 4 감지 → LLM 완료, 씬 슬롯 4개 렌더링 시작
4. 각 씬의 status === "End" 감지 → 해당 씬 재생 가능
5. scenarios 배열의 index 순서대로 재생:
  - image\_url 있으면 이미지 표시 후 delay초 대기
  - audio\_url 있으면 오디오 재생, 완료까지 대기 (delay: 0)
  - msg 있으면 자막 표시
6. status === "DONE" → 전체 완료

## 씬 재생 구현 예시

```

async function playScene(scene) {
  for (const step of scene.scenarios) {
    if (step.image_url) {
      showImage(step.image_url);
      await wait(step.delay * 1000);
    }
    if (step.audio_url) {
      showSubtitle(step.msg);
      await playAudio(step.audio_url); // 재생 완료까지 대기
    }
    if (!step.image_url && !step.audio_url && step.msg) {
      showSubtitle(step.msg); // index 4: 대사 자막만 표시
    }
  }
}

```

## 진행률 표시 예시

```
// 씬 수는 항상 4개 고정
const total = 4;
const done = scenes.filter(s => s.status === "End").length;
// `${done} / ${total} 씬 준비됨`
```

## curl 예시

### 작업 생성

```
curl -X POST http://127.0.0.1:8000/api/runs \
-H "Content-Type: application/json" \
-d '{
  "era_ko": "조선 시대",
  "place_ko": "한양",
  "characters_ko": "호기심 많은 아이와 말하는 호랑이",
  "topic_ko": "우정"
}'
```

### 상태 조회

```
curl http://127.0.0.1:8000/api/runs/20260429_173423_d118aa
```

### SSE 확인

```
curl -N http://127.0.0.1:8000/api/runs/20260429_173423_d118aa/events
```

### 산출물 다운로드

```
curl -o cover.png
http://127.0.0.1:8000/api/runs/20260429_173423_d118aa/images/cover.png
curl -o cover.wav
http://127.0.0.1:8000/api/runs/20260429_173423_d118aa/audio/cover.wav
curl -o scene_01.wav
http://127.0.0.1:8000/api/runs/20260429_173423_d118aa/audio/scene_01.wav
curl -o story.json http://127.0.0.1:8000/api/runs/20260429_173423_d118aa/story
```